

```

import pandas as pd
import numpy as np

def load_data(filepath: str) -> pd.DataFrame:
    """Load and clean the arctic ice extent dataset, and enrich with derived feat
    df = pd.read_csv(filepath)

    # Clean column names and values
    df.columns = df.columns.str.strip()
    df["extent"] = df["extent"].astype(str).str.strip().astype(float)
    df["year"] = df["year"].astype(int)

    # --- Derived / Engineered Features ---
    df["decade"] = (df["year"] // 10 * 10).astype(str) + "s"
    df["change_from_prev"] = df["extent"].diff().round(3)
    df["rolling_avg_5yr"] = df["extent"].rolling(window=5, min_periods=1).mean().
    df["pct_change"] = df["extent"].pct_change().mul(100).round(2)

    # Classify each year's extent relative to overall median
    median_ext = df["extent"].median()
    df["extent_category"] = df["extent"].apply(
        lambda x: "Above Median" if x >= median_ext else "Below Median"
    )

    # Trend era classification
    def era(y):
        if y < 1990:
            return "Pre-1990"
        elif y < 2000:
            return "1990s"
        elif y < 2010:
            return "2000s"
        else:
            return "2010s+"
    df["era"] = df["year"].apply(era)

    # Season analogy: odd/even year (proxy for oscillation pattern)
    df["year_parity"] = df["year"].apply(lambda y: "Odd Year" if y % 2 != 0 else

    # Anomaly: deviation from long-term mean
    mean_ext = df["extent"].mean()
    df["anomaly"] = (df["extent"] - mean_ext).round(3)

```

```

return df

def apply_filters(df: pd.DataFrame,
                  year_range: tuple,
                  selected_eras: list,
                  extent_range: tuple,
                  selected_categories: list,
                  search_text: str) -> pd.DataFrame:
    """Apply all sidebar filters and return filtered DataFrame."""
    filtered = df.copy()

    # Year range filter
    filtered = filtered[
        (filtered["year"] >= year_range[0]) & (filtered["year"] <= year_range[1])
    ]

    # Era multi-select filter
    if selected_eras:
        filtered = filtered[filtered["era"].isin(selected_eras)]

    # Extent numerical range slider
    filtered = filtered[
        (filtered["extent"] >= extent_range[0]) & (filtered["extent"] <= extent_r
    ]

    # Category multi-select filter
    if selected_categories:
        filtered = filtered[filtered["extent_category"].isin(selected_categories)]

    # Search / text filter (year or era contains search string)
    if search_text.strip():
        mask = (
            filtered["year"].astype(str).str.contains(search_text.strip(), case=F
            filtered["era"].str.contains(search_text.strip(), case=False) |
            filtered["decade"].str.contains(search_text.strip(), case=False)
        )
        filtered = filtered[mask]

    return filtered

```